

SiamRAM

A Theoretical Account of the Robustness Layer Built on the SiamABC Visual Tracker

Distractor-aware re-acquisition, occlusion recovery, motion filtering, and camera-motion compensation — explained from first principles.

Document	Theory & architecture report (reader-facing, not a code walkthrough)
Base tracker	SiamABC — <i>Improving Accuracy and Generalization for Efficient Visual Tracking</i> (Zaveri et al.,
System	SiamRAM — long-term tracker wrapping SiamABC in a state machine of safety nets
Scope	What we added on top of SiamABC, and <i>why</i> each piece exists

Abstract

SiamABC is a fast, accurate Siamese tracker: given the target's last position it localises the target in a small search window, adapting to appearance change through a dynamic template and a backward-free test-time adaptation rule. It is, by design, a *local appearance optimiser* — and that is exactly its blind spot. It has no concept of identity beyond correlation, no memory of where the target went when it disappears, and no defence against a look-alike that briefly scores higher. **SiamRAM** keeps SiamABC as the per-frame engine and wraps it in a layer that supplies the three things a pure local tracker lacks: a *motion model* that knows where the target should be, an *identity memory* that knows what the target looks like over time, and a *state machine* that decides — every frame — whether to trust the fast tracker, arbitrate against a distractor, or launch a structured re-detection. This report explains each added component theoretically: the bounding-box Kalman filter, the camera-motion / global-motion-compensation subsystem, the two-tier appearance memory (RAM + DRM) with its composite re-acquisition score, the adaptive controllers that auto-tune the system per video, and the three behavioural modes — normal, distractor, and occlusion — that orchestrate them.

Contents

1. Introduction: the problem and the thesis
2. Background: SiamABC, the foundation we extend
3. From local tracker to robust system: the design model
4. The motion model — a bounding-box Kalman filter
5. Camera-motion estimation and the GMC search prior

6. Appearance memory — RAM, DRM, and the distractor bank
7. Adaptive controllers — auto-tuning the system per video
8. The master per-frame state machine
9. Occlusion recovery — structured re-detection
10. Distractor mode — identity arbitration
11. Spike rejection — the trigger that guards identity
12. Synthesis: how the pieces compose, and the design principles
13. Parameter reference

PART 1

Introduction: the problem and the thesis

Single-object tracking states a deceptively simple task: *here is a bounding box on frame zero; follow this exact object for the rest of the video*. When the target stays visible, sharp, and alone, a fast Siamese tracker solves it almost trivially — it correlates a stored template against the current frame and reads off the peak. The difficulty is entirely in the exceptions, and there are three of them.

When the target...	A naive Siamese tracker...	SiamRAM adds...
is briefly hidden or leaves the frame	drifts onto the background and never recovers	Occlusion mode — a structured re-detection pipeline (Part 9)
has a look-alike nearby (another player, another car)	silently snaps onto the impostor and never returns	Distractor mode — active identity arbitration (Part 10)
jitters or jumps for a single frame (blur, partial occlusion)	locks onto the jump and memorises the wrong object	Spike rejection — jump detection that feeds distractor mode (Part 11)

These are not three unrelated bugs; they are three different ways the same assumption fails. A pure appearance tracker assumes *the best-correlating patch near the last position is the target*. That assumption breaks when the target is not near the last position (occlusion), when a different object correlates better (distractor), or when a single corrupted frame produces a spurious peak (spike). SiamRAM's thesis is that each failure needs a *different* response — and that conflating them into one "low-confidence" branch is what makes naive trackers brittle.

The one-sentence thesis

SiamRAM is a **fast local tracker wrapped in a state machine of safety nets**: SiamABC localises the target every frame, while a motion model, a camera-motion estimator, a two-tier appearance memory, and three behavioural modes watch it and take over the instant its confidence collapses or its motion stops making physical sense.

This document is deliberately a *theory* report, not a code tour. It explains what each added component computes and, more importantly, the reasoning that makes it the right thing to compute. Where a design choice looks arbitrary, there is almost always a concrete failure it was built to prevent; those failures are called out throughout. A compact parameter reference closes the report (Part 13) for when you move from understanding to tuning.

A note on configuration. Every behaviour described here is a capability of the system; which behaviours are *active* is set by the inference configuration. The current leaderboard configuration, for example, keeps the spike/distractor machinery present but disabled, and runs the re-acquisition memory in an appearance-dominant regime. These choices are flagged where they matter so the reader never confuses "the system can do X" with "the system is doing X right now."

PART 2

Background: SiamABC, the foundation we extend

SiamRAM does not modify SiamABC's network; it consumes it. To understand what the wrapper adds, it helps to know precisely what the base tracker already provides — and where it stops. This part is a compact recap of SiamABC; readers familiar with the paper can skim to Part 3.

2.1 The dual-template, dual-search-region idea

Classic Siamese trackers compare a single fixed template (the target on frame 0) against the current search region. SiamABC maintains *four* image sources and bridges the time gap between them:

- **Static template** I_T — a crop of the target at $t = 0$. Never changes; the ground-truth identity anchor.
- **Dynamic template** I_D — the target at a recent time $t - \Delta t$, refreshed during tracking to absorb gradual change in pose, lighting, and scale.
- **Search region** I_t — where the target is presumed to be now.
- **Dynamic search region** I_S — the search region at $t - \Delta t$, re-centred on the target (so it contains I_D).

The static template anchors identity at $t = 0$ (point A); the dynamic pair represents $t - \Delta t$ (point B); the search region represents now (point C). The network learns relations $A \rightarrow B$ (template bridge) and $B \rightarrow C$ (search bridge), then blends them — hence the name SiamABC. This is what lets the base tracker survive appearance change without forgetting the original target.

2.2 Fast Mixed Filtration and the relation-aware block

Each crop is encoded by a shared backbone. The template pair and the search pair are then each fused not by naive concatenation but by a learnable **Fast Mixed Filtration (FMF)** layer — a lightweight separable attention that applies a channel filter and a spatial filter and combines them, all with broadcasted element-wise operations so it stays cheap on CPU:

$$\check{x} = \Omega(F_a, F_b) = \text{Conv}_{2C \rightarrow C}((A_{ch}(x) \oplus A_{sp}(x)) \odot x), \quad x = F_a \cup F_b$$

The filtered template representation and filtered search representation are combined by a pixel-wise cross-correlation, then passed to a classification head (is the target here?) and an anchor-free regression head (where exactly?) that predict on a 16×16 grid. The regression is FCOS-style: each grid cell predicts four distances to the box edges, giving sub-cell, scale-flexible boxes without anchor tuning. A cosine window and a scale/aspect penalty bias the score map toward the centre and toward boxes similar in size to the previous one — the mechanism that keeps a Siamese tracker from teleporting frame to frame.

2.3 Transitive relation loss and dynamic test-time adaptation

Two ideas from SiamABC matter for what follows. First, the **transitive relation loss** trains the filtered template and search representations to be mutually consistent (they all describe the same object), which is what makes the base features usable as an identity signal at all. Second, and more directly relevant, SiamABC introduces a backward-free **dynamic test-time adaptation (DTTA)**: during inference it nudges the batch-norm statistics of the heads toward the current instance while remaining anchored to the source statistics,

$$\mu_{l,t} = (1 - \lambda_{BN}) \bar{\mu} + \lambda_{BN} \mu_{l,t}, \quad \sigma_{l,t}^2 = (1 - \lambda_{BN}) \bar{\sigma}^2 + \lambda_{BN} \sigma_{l,t}^2$$

with the source statistics acting as a prior so the model adapts to out-of-distribution video without drifting away from what it learned. SiamABC also refreshes its dynamic template on a running-average-of-scores schedule. SiamRAM controls both of these — it *freezes* the template update and DTTA the moment a target is declared lost, so the tracker can never quietly memorise the occluder, and re-enables them on recovery. That freeze/thaw control is one of the simplest but most important couplings between the wrapper and the base tracker.

Where SiamABC stops

SiamABC is a track-by-similarity **local optimiser**: fast and precise while identity is stable, self-adapting through its dynamic template. It has no motion model, no long-term memory, and no notion of "a different object that looks similar." Everything in the rest of this report exists to supply exactly those missing faculties.

PART 3

From local tracker to robust system: the design model

Before the components, the architecture. SiamRAM is organised as one integration hub that owns the base tracker and delegates to a set of single-responsibility subsystems. The hub holds the *state* — which mode we are in, the held position, the memory — and the subsystems hold the *logic* for each concern.

```
SiamRAM tracker (the integration hub / state machine)
■■ SiamABC tracker      the fast local engine      (Part 2)
■■ Bounding-box EKF     constant-velocity motion   (Part 4)
■■ Camera-motion subs.  homography / GMC search prior (Part 5)
■■ Appearance memory    RAM short-term + DRM long-term (Part 6)
■■ Adaptive controllers auto margin, auto template rate (Part 7)
■■ Spike watcher        single-frame jump detection (Part 11)
■■ Distractor subsystem identity arbitration        (Part 10)
■■ Occlusion subsystem  re-detection phases         (Part 9)
■■ YOLO re-detector     the detector used by 9 and 10
```

3.1 One frame, one decision

The single most important structural fact is that **exactly one of two update paths runs per frame**: the normal update or the occlusion update. They are different worlds. While in occlusion the tracker's public output is a placeholder and the real position estimate is carried internally by the motion model; normal coordinates resume only on the frame recovery commits. Within the normal path, distractor arbitration and spike detection are *mutually exclusive arms*: if you are already arbitrating a distractor you run that; otherwise you run spike detection, which is the thing that can *enter* distractor mode. The high-level loop is:

```
for each frame:
  prescale; estimate camera homography H; EKF.predict(H)
  if in_occlusion: run OCCLUSION machine      (Part 9)
  else:           run NORMAL update:
    bbox, score = SiamABC.update(frame)
    (early) class + YOLO-detectability probes
    if distractor_mode_active: arbitrate      (Part 10)
    else:                     spike-check     (Part 11)
    if score low for entry_patience frames: ENTER occlusion
    else: commit frame; EKF.update; admit to memory
```

3.2 The two coordinate principles

Two conventions make the rest coherent. First, all internal work happens at a bounded *processing resolution* (the frame is downscaled so its long edge is at most a fixed size); boxes are scaled back only at the output boundary. This keeps latency predictable on large video. Second, the *held box* — the motion model's best estimate of the target — is maintained continuously even while the public output is suppressed during occlusion. The held box is what the re-detection ROI grows around, so the system always has a physically grounded guess of where to look.

With that scaffold in place, the next four parts describe the building blocks bottom-up — motion, camera, memory, adaptation — and the final four describe the state machine and the three modes that coordinate them.

PART 4

The motion model — a bounding-box Kalman filter

SiamABC knows appearance; it does not know *motion*. When the target vanishes there is no appearance signal at all, yet the system must still answer "where is it likely to be?" That answer comes from a small Extended Kalman Filter on the bounding-box centre. It is the quiet workhorse of the whole system: it drives the held box during occlusion, the search centre for re-detection, the velocity used in candidate scoring, and the uncertainty that widens spatial tolerances when the system is unsure.

4.1 State, measurement, and the one deliberate omission

The filter tracks a constant-velocity model of the centre:

$$\mathbf{x} = [c_x, c_y, v_x, v_y]^\top, \quad \mathbf{z} = [c_x, c_y]^\top$$

Conspicuously, **width and height are not in the state**. They are smoothed entirely separately, by a slow exponential moving average:

$$w_t = (1 - \beta) w_{t-1} + \beta w_t^{\text{meas}}, \quad h_t = (1 - \beta) h_{t-1} + \beta h_t^{\text{meas}}, \quad \beta \approx 0.15$$

Why size is kept out of the filter — occlusion-shrinkage immunity

When a target is being occluded, the tracker's box very often collapses to a sliver in the last frames before loss (it is latching onto the visible fragment). If size were coupled into velocity, that collapse would inject a huge spurious velocity *exactly* at the moment you most need a clean extrapolation of where the target is going. By estimating only the centre, velocity stays physically meaningful right through the moment of loss; size is recovered separately and is allowed to be wrong without poisoning the trajectory.

4.2 Prediction that separates target motion from camera motion

The prediction step is where the EKF earns the "Extended" in its name. If a reliable camera homography H is available for this frame (Part 5), the centre is first warped by H — the apparent motion induced by the camera — and only then advanced by the target's own velocity:

$$\hat{\mathbf{p}}_t = \pi(H[c_{x,t-1}, c_{y,t-1}, 1]^\top) + \Delta t[v_x, v_y]^\top$$

Camera warp of the previous centre (left term), then advance by the target's own velocity (right term).

where π denotes the perspective division back to image coordinates. The Jacobian of the transition is taken *through* the homography, so the covariance grows correctly under the nonlinear warp. Without a reliable H the step degrades gracefully to plain constant velocity. The payoff is conceptual as much as numerical: the filter can now say "*the target did not move in the world; the camera panned*," instead of inventing target velocity to explain ego-motion. That distinction is what prevents false losses during hard pans.

4.3 Three transitions the modes rely on

- **update(z)** — the standard Kalman correction from a measured centre, used every healthy frame and at recovery commit.

- **reseed(bbox, v)** — a hard reset of position *and* velocity with inflated covariance. Used at occlusion entry, seeded from clean history (Part 9.1) so the post-loss extrapolation reflects healthy motion rather than the drift into loss.
- **nudge_position(bbox)** — move the centre toward a near-miss detection *without* touching velocity. Used to gently steer the held box toward a nearby YOLO box during a failed recovery, so the search ROI follows real evidence without committing to it.

Finally, the filter exposes its positional uncertainty as a scalar:

$$u_t = \sqrt{\frac{1}{2}(P_{c_x c_x} + P_{c_y c_y})}$$

This single number is reused widely: it sets the spatial tolerance σ in the re-acquisition score (Part 6.4) and the extra padding on the distractor ROI (Part 10). The principle is that **spatial gates should widen exactly when the filter admits it is unsure** — a uniform tolerance is either too tight after a long occlusion or too loose during confident tracking.

PART 5

Camera-motion estimation and the GMC search prior

Everything in this part serves one goal: **separate target motion from camera motion**, so that loss detection, velocity, and search placement stay physically plausible during pans, shakes, and zooms. A tracker that cannot tell a target moving left from a camera panning right will declare false losses and chase phantom velocities.

5.1 Estimating the inter-frame homography

Each frame, the subsystem estimates the background motion between the previous and current frame as a homography H together with a strict **reliability** flag. Three estimators trade speed for robustness:

- **classic** — a grid of corners, optical flow, and affine RANSAC on a downsampled frame. Fast; the default.
- **accurate** — feature matching and a full homography, falling back to classic if the fit fails.
- **botstort** — a BoT-SORT-inspired global-motion-compensation estimator with strong validity gates (inlier ratio, residual robustness, translation/scale/rotation bounds, spatial coverage) and a dynamic exclusion mask around the target so the *target's* own motion never contaminates the *background* estimate.

The reliability flag is intentionally conservative: the fit must pass every validity gate and must not be a fallback transform. Downstream consumers are written to treat an unreliable H as "no camera-motion information," never as "zero camera motion" — a distinction that keeps the system honest when the scene is too textureless or too fast to estimate.

5.2 The GMC search prior

A reliable homography is not just fed to the EKF; it is used to *move SiamABC's search window before the network runs*. If H passes a plausibility check — bounded translation, scale, rotation, and corner displacement — the previous box is warped forward by H and handed to SiamABC as its search-centre prior:

$$\mathbf{b}_t^{\text{prior}} = H \cdot \mathbf{b}_{t-1} \quad \text{if valid}(H); \quad \text{else } \mathbf{b}_t^{\text{prior}} = \mathbf{b}_{t-1}$$

The effect: during a hard pan, SiamABC starts looking where the target *will* be, not where it *was*, so the target never slides out of the search crop between frames. The prior is deliberately skipped while in distractor mode, to keep that arbitration logic independent of camera estimation.

5.3 The heavy-motion signal and its three consumers

From the same estimate the subsystem derives a single boolean — *is camera motion heavy right now?* — by normalising displacement against a reference diagonal and tripping when either an absolute pixel gate or a normalized gate is exceeded:

$$\text{heavy} : \quad d_{\text{px}} \geq \tau_{\text{px}} \quad \text{or} \quad \frac{d_{\text{px}}}{\text{diag}} \geq \tau_{\text{norm}}$$

The normalized gate matters for tiny targets, where a few pixels of camera shake is enormous relative to the box. This one boolean is consumed in three places, each a guard against the camera fooling a different subsystem:

- 1 **Block occlusion entry** during heavy motion — a transient low score during a pan is usually the tracker lagging, not a real loss.

- 2 **Block distractor-mode entry** during heavy motion — an apparent jump during shake is usually ego-motion, not a genuine identity switch.
- 3 **Swap the dynamic-template adaptation parameters** — under heavy motion the template cadence, window, and admission gates can temporarily change so the tracker adapts appropriately through blur, with a linger period after the motion calms.

Design principle: distrust the camera before distrusting the tracker

Each of the three consumers encodes the same prior — when the camera is moving hard, surprising tracker behaviour is more likely an artefact of ego-motion than a real event. Rather than react and recover, SiamRAM *suppresses* the reaction until the camera settles. This is cheaper and far less error-prone than entering and then aborting a recovery.

PART 6

Appearance memory — RAM, DRM, and the distractor bank

Re-detection is only as trustworthy as the system's memory of what the target looks like. SiamRAM maintains a structured **identity memory** with two positive tiers and one negative bank. A *descriptor* is an appearance embedding of a box; the default backend is an OSNet re-ID network compared by cosine similarity, so the question "are these the same object?" reduces to a similarity in $[-1, 1]$.

6.1 RAM — the conservative short-term record

RAM (recent appearance memory) is a rolling buffer of (box, descriptor) pairs from recent confident frames. A frame is admitted only if it passes two continuity gates:

$$\text{IoU}(b_t, b_{t-1}) \geq \tau_{\text{IoU}} \quad \text{and} \quad |\text{area}(b_t) - \tilde{a}| \leq \tau_{\text{area}} \cdot \tilde{a}$$

where the tilde denotes the buffer's median area. In words: RAM only remembers frames where the target moved smoothly and kept a sensible size. This is intentionally strict — RAM is the clean source material from which long-term memory is distilled, so admitting a jump or a size glitch would poison everything downstream.

6.2 DRM — the resilient long-term bank

The Dynamic Reference Memory (DRM) is the long-term identity-plus-motion prior used to recover after loss. A RAM entry is *promoted* to DRM only if, within a recent window, at least $m_{\min}$ stored descriptors independently agree with it (cosine $\geq \tau_{\text{sim}}$):

$$|\{k \in W : \cos(d_{\text{new}}, d_k) \geq \tau_{\text{sim}}\}| \geq m_{\min} \Rightarrow \text{promote to DRM}$$

This is a vote, not a snapshot: a single lucky frame can enter RAM, but it cannot enter DRM unless several recent frames corroborate it as the same object. DRM is therefore robust to one-off appearance flukes — precisely the entries you do *not* want to rely on when deciding whether a re-detected box is really the target.

6.3 The DRM composite score — the heart of re-acquisition

When the target is lost and YOLO proposes candidate boxes, DRM ranks them with a composite score that fuses five sources of evidence. For a candidate c with descriptor d_c , it is scored against every DRM anchor k and takes the best-supporting anchor:

$$\text{anchor}(k) = \lambda_{\text{IoU}} \text{IoU}(b_k, c) + \lambda_{\text{app}} \cos(d_k, d_c) + \lambda_{\text{mot}} \max(0, \cos(\mathbf{v}, \mathbf{p}_k - \mathbf{p}_{\text{ref}})) + \lambda_{\text{time}} e^{-\alpha \text{age}_k} - \gamma \max_n \cos(d_c, d_n)$$

$$\text{score}(c) = \max_k \text{anchor}(k) - \lambda_{\text{dist}} (1 - e^{-\frac{1}{2}(d/\sigma)^2}) + \lambda_{\text{dir}} \cos(\mathbf{v}, c - \mathbf{p}_{\text{ref}})$$

The five terms each answer a distinct question:

- **Appearance** $\cos(d_k, d_c)$ — does it look like the target? (the dominant signal)
- **IoU** — does it overlap where an anchor says the target was?
- **Motion** — is it moving in the direction the velocity predicts? (rectified, so wrong-way motion adds nothing rather than penalising)
- **Time** $e^{-\alpha \cdot \text{age}}$ — recent anchors weigh more than stale ones.

- **Negative** $-\gamma \cdot \max \cos(d_c, \text{distractor bank})$ — explicitly subtract similarity to known impostors.

Candidates above the acceptance *margin* are kept and sorted; if the best clears a skip threshold it is returned alone, otherwise the top- k proceed to SiamABC verification. The composite is, in effect, a hand-built scoring function standing in for the question *"is this YOLO box really our target, given everything we know — appearance, motion, recency, geometry, and what the impostors look like?"*

Configuration note — the appearance-dominant regime

The composite supports all five terms, but the current leaderboard configuration sets $\lambda_{\text{app}} = 1$ and the other weights to 0, with the acceptance margin on "auto" (Part 7.1). In that regime re-acquisition is driven by appearance similarity against a per-video-calibrated bar, with the geometric and motion terms available but dormant. The machinery is general; the operating point is a tuning choice.

6.4 Uncertainty-aware spatial tolerance

The distance penalty's width σ is not a constant — it is tied to the EKF's positional uncertainty (Part 4.3). After a long occlusion the filter is uncertain, σ widens, and the score tolerates candidates farther from the predicted centre; during confident tracking σ tightens. This closes the loop between the motion model and the memory: **how far afield to trust an appearance match depends on how confident the motion model is about position.**

PART 7

Adaptive controllers — auto-tuning the system per video

A single fixed threshold rarely fits every video. A low-texture, small, or fast target genuinely scores lower than a large, distinctive one; a steady scene wants slow, stable template averaging while a fast scene wants quick snapping. SiamRAM addresses this with small, self-contained controllers that calibrate two key dials online. Both are intentionally decoupled from the tracker internals — they consume scalars and emit scalars — so the legacy fixed-value paths are byte-for-byte unchanged when the controllers are off.

7.1 Adaptive DRM acceptance margin

The DRM gate accepts a re-acquisition when the composite score exceeds a *margin*. The insight behind the adaptive margin is that the right bar is *just below where the genuine target itself scores*. So during healthy tracking the system continuously measures the true target's self-score against its own DRM anchors — using the exact same composite formula, so the numbers are on the same scale — and tracks it with an EMA:

$$\bar{s}_t = (1 - a) \bar{s}_{t-1} + a s_t^{\text{self}}, \quad \text{margin}_t = \text{clip}(\bar{s}_t - \delta, m_{\min}, m_{\max})$$

The margin sits a small δ below the EMA of the genuine self-score, clamped to a safe range, and only begins adapting after a minimum number of samples (before which it behaves as a fixed warm-up margin). The consequence is elegant: on a distinctive target the bar rises (be picky — you can afford to); on a hard, low-scoring target the bar lowers (don't reject the real re-acquisition just because it scores modestly). A margin tuned for one sequence no longer either rejects valid recoveries or admits distractors on another.

7.2 Adaptive dynamic-template update rate

SiamABC refreshes its dynamic template every N frames over a rolling window. The right (N , window) pair depends on motion: a steady camera with a slowly moving target wants *large* N and a *large* window (lots of averaging, very stable); fast camera or target motion wants *small* N and a *small* window (snap to the latest look). The controller measures a normalized "fastness" in $[0, 1]$ from camera and target motion — combined with a max so that *either* being fast drives fast updating — smooths it, and linearly interpolates both dials:

$$m_t = (1 - a) m_{t-1} + a \max(\hat{c}_t, \hat{g}_t), \quad m_t \in [0, 1]$$

$$N_t = \text{lerp}(N_{\text{slow}}, N_{\text{fast}}, m_t), \quad W_t = \text{lerp}(W_{\text{slow}}, W_{\text{fast}}, m_t)$$

where $\hat{c}$ and $\hat{g}$ are the normalized camera and target speeds. At $m = 0$ the dials sit at the slow, stable end; at $m = 1$ they snap to the fast end; everything between glides smoothly. A simpler binary variant exists too — a hard slow/fast switch on a target-speed pixel threshold — and a strict precedence orders the three template-adaptation schemes so they never fight over the same dials.

The unifying idea behind both controllers

Both replace a brittle constant with a quantity the system can *measure about itself*: the genuine target's own score sets the acceptance bar, and the observed motion sets the adaptation speed. This is the same self-referential principle as SiamABC's test-time adaptation, lifted from batch-norm statistics up to the tracker's decision thresholds.

PART 8

The master per-frame state machine

The components so far are inert until something decides when to use them. That is the job of the state machine, governed by three flags — *in occlusion*, *distractor active*, and the spike watcher's *jump watch* — and a strict per-frame decision order.

8.1 The decision order

After prescaling, estimating H , and running the EKF prediction, the frame is routed:

- 1 **If already in occlusion** → run the occlusion machine (Part 9) and return; the normal path does not run at all this frame.
- 2 **Otherwise run the normal update:** SiamABC produces (box, score); early frames also run the class-vote and YOLO-detectability probes.
- 3 **If distractor mode is active** → run distractor arbitration (Part 10). **Else** → run spike detection (Part 11), which may itself enter distractor mode this frame.
- 4 **Loss test:** if the score has stayed below the effective threshold for *entry_patience* frames (and heavy camera motion is not blocking), enter occlusion.
- 5 **Otherwise it is a healthy frame:** correct the EKF, update velocity from history, admit the descriptor to RAM, and commit the frame.

Mutual exclusivity

Distractor arbitration and spike detection are the two arms of an if/else — they never run in the same frame. Spike detection is the *sensor* that decides a switch happened; distractor mode is the *controller* that sorts it out. One hands off to the other; they do not coexist.

8.2 The effective threshold is adaptive

"Loss" is not judged by one fixed score. The effective threshold depends on the target's apparent size: a far, tiny target (a few-pixel UAV) uses a dedicated long-distance confidence threshold and a wider search context than a large foreground object. This matters because small targets legitimately produce lower correlation peaks; holding them to a foreground target's bar would declare constant false losses.

8.3 The hysteretic entry, in brief

Entry into occlusion is never instantaneous. A low-score frame increments a streak; a confident frame resets it; a heavy-camera-motion guard can hold the streak at zero during pans (unless the target is genuinely exiting the frame); and a warm-up grace period ignores the unsettled early frames entirely. Only a *sustained* low score crosses into occlusion. This hysteresis is the difference between a tracker that survives a one-frame blur and one that panics at it.

PART 9

Occlusion recovery — structured re-detection

Goal. The target has been lost — hidden, or gone off-frame. Find it again, confidently, without locking onto a distractor or the background — and while it is lost, hold a physically plausible position estimate. Occlusion is a separate world with its own phase counter; the public output is a placeholder until recovery commits, while the held box silently tracks the EKF.

9.1 Entry: declaring loss, and rebuilding clean motion

When the hysteretic test fires, entry does substantially more than flip a flag. It classifies the *loss cause* and the *exit direction* (setting an out-of-frame flag and exit edge if the target left the frame); it freezes learning — dynamic-template updates off, test-time adaptation off — so the tracker can never memorise the occluder; and, crucially, it **rebuilds the EKF from clean history**:

$$\text{skip} = \max(\text{skip}_{\text{drift}}, \text{streak}_{\text{entry}})$$

The last few frames before a loss are usually corrupted (the box was already drifting or collapsing). Those frames are skipped, and the filter is re-seeded from the healthy frames before them, so the velocity used to extrapolate where the target went reflects real motion — not the death-spiral into loss. For out-of-frame and camera-motion causes the skip is forced to zero, since those losses are abrupt rather than gradual. The entry frame is itself the first recovery frame.

9.2 Per-frame housekeeping and the out-of-frame state machine

Every occluded frame, before any phase logic: the EKF predicts, updating the held box and the search centre and refreshing velocity. A small state machine handles targets that left the frame — if off-screen, the search ROI is *pinned to the exit edge* rather than centred on a prediction that points off-frame, and the target is declared back in-frame only when the predicted centre is inside *and* velocity points inward. This prevents the absurd case of searching empty space outside the image.

9.3 The three recovery phases

Recovery is routed by a phase counter into three escalating strategies:

- **Phase 0 — fast SiamABC reacquire.** The cheap path: grow a square search ROI around the predicted centre, plant a seed box at the median target size, run one SiamABC pass, and require *both* a SiamABC score gate (Gate A) *and* a DRM appearance gate (Gate B) before committing. No YOLO. If either gate fails, advance to the next phase as the detectability policy dictates.
- **Phase 1 — candidate collection.** Run YOLO in the growing ROI across several frames, extract descriptors, and store (box, descriptor) per frame while recording camera velocity. Collecting across multiple frames is what makes per-candidate *velocity* measurable. Any detection heavily overlapping the held box is added to the distractor bank — it sits exactly where we lost the target, so it is a prime impostor suspect.
- **Phase 2 — final DRM verification.** Trace each candidate's velocity backward through the collection frames (camera-compensated; candidates that cannot be matched across frames are dropped for lacking a reliable velocity), rank by the DRM composite (Part 6.3) with the uncertainty-aware σ , augment with a direction bonus (zeroed off-frame, halved for tiny targets), then motion-compensate the top- k and verify each with SiamABC. The first candidate whose verification clears the re-acquisition threshold wins.

9.4 The adaptive detectability policy

A class-agnostic probe runs *early, during normal tracking*: every few frames it runs YOLO on the tracker's search ROI and counts a "hit" when a detection overlaps the tracked box. Enough hits → the target is **YOLO-detectable**; attempts exhausted without hits → **not detectable**. That single verdict, decided long before any loss, rewrites the recovery strategy:

Verdict	What it means	Recovery strategy
Detectable	YOLO reliably finds this object class	Skip the Phase-0 SiamABC commit; rely on YOLO + DRM (Phases 1–2), which is strictly stronger and avoids premature commits.
Not detectable	YOLO never found it (unusual class / appearance)	Lean entirely on Phase 0 (SiamABC + DRM gates), retrying every frame; YOLO collection is pointless and is skipped.
Unknown / disabled	Loss began before the probe finished, or feature off	Legacy flow: Phase 0 may commit; on failure fall through to collection + DRM.

The reasoning is that the two recovery engines have different strengths, and the system should know *before* it is in trouble which engine to trust. Probing a class the detector cannot see would waste every recovery frame on futile YOLO calls; committing on SiamABC alone for a class YOLO can verify would risk a premature lock onto a look-alike. The probe lets recovery pick the right tool up front.

9.5 Confirmation, commit, and the no-give-up stance

A verified candidate may optionally be required to re-verify for several consecutive frames before committing — a single miss restarts the cycle — which guards against a one-frame fluke. **Commit** is the single exit point for every recovery path: it corrects the EKF to the recovered box, clears all occlusion and distractor episode state, un-freezes the template and test-time adaptation, admits the recovered descriptor to memory, and resumes normal tracking on the next frame.

There is no timeout

If no candidate verifies, the held box keeps following the EKF (nudged toward the nearest YOLO box when one exists) and the cycle resets to Phase 0. Occlusion persists until something verifies — a deliberate "never give up the identity" stance. The system would rather hold a motion-extrapolated estimate indefinitely than hand control back to a base tracker that may be confidently sitting on the wrong object.

PART 10

Distractor mode — identity arbitration

Goal. The tracker is at risk of confusing the target with a nearby look-alike. Keep the *right* identity by actively arbitrating between candidates in a focus region — using appearance, motion, and explicit negative memory — and refuse to switch unless the evidence is strong. Where occlusion answers "where did it go?", distractor mode answers "which of these similar objects is mine?"

10.1 Entry and the per-frame arbitration

Distractor mode is entered by the spike watcher (Part 11) after it confirms a jump-switch and snaps back to a stable pre-jump anchor. Entry seeds a *focus box* at the anchor and spins up a dedicated anchor EKF reseeded from the main filter's velocity. Each frame thereafter: advance the anchor EKF to a focus centre with uncertainty padding, build an ROI around it, run YOLO, and score every candidate against the recent target descriptors:

$$\text{score}(c) = \lambda_{\text{app}} \max_r \cos(d_c, d_r) + \lambda_{\text{iou}} \text{IoU}(c, \text{focus}) - \lambda_{\text{dist}} (1 - e^{-\frac{1}{2}(d/\sigma)^2}) - \gamma \max_n \cos(d_c, d_n)$$

A Mahalanobis motion gate drops candidates whose centre is statistically implausible under the anchor EKF (the χ^2 test below). The winner is the arg-max; *everything else in the ROI is, by construction, labelled a distractor* and its descriptor is added to the negative bank — so the system actively learns what the impostors look like as it goes.

$$(\mathbf{c} - \hat{\mathbf{p}})^\top \mathbf{S}^{-1} (\mathbf{c} - \hat{\mathbf{p}}) > \chi_{\text{thr}}^2 \Rightarrow \text{reject candidate}$$

10.2 Four ways the winner is not immediately accepted

This is the heart of distractor mode: a cascade of guards that bias hard toward *not* switching identity on weak evidence.

- 1 Below-gate hold.** If the top candidate's appearance dips under the accept gate (blur, pose, partial occlusion), do not abandon the target — which would hand control back to a base tracker often sitting on the distractor — but ride the EKF/anchor motion prediction for a bounded number of frames and wait for appearance to recover. The counter resets the instant the winner clears the gate.
- 2 Overlap motion-lock.** When the winner heavily overlaps a known distractor, the two are visually fused; rather than flicker between them, hold the EKF-predicted box until the overlap clears for several frames (or a hard cap elapses).
- 3 Ambiguity hold.** If the top-1 vs top-2 margin is too small, stick with the current pick rather than switch on a coin-flip.
- 4 Stable-exit requirement.** Even a clean winner must agree with the focus box for several consecutive frames before the mode is allowed to resolve.

If the below-gate hold is exhausted (the target is presumed truly lost), the mode can **force an immediate handoff to occlusion recovery** rather than revert to a base tracker that may be confidently wrong. That handoff tags the occlusion episode as *distractor-origin*, which selects a separate set of DRM recovery weights — a recovery biased to avoid re-locking the very distractor that just stole the target.

10.3 Exit and contamination control

On a confident, stable resolution the mode optionally re-initialises the *dynamic* template only (refreshing recent appearance while preserving the frame-0 static anchor) and exits. Resolution arms three guards, because the moment just after a distractor episode is exactly when a wrong descriptor would poison long-term memory:

- **Re-entry cooldown** — cannot re-enter distractor mode immediately.
- **Memory freeze** — pause appearance-memory admission for a few frames.
- **Template freeze** — pause dynamic-template updates for a few frames.

Even outside distractor mode, for a window after any jump the score is penalised if the current box looks like a known distractor or drifts far from the focus point — making it harder for a recently-seen impostor to win the base tracker's confidence right after a scare.

Configuration note — distractor/spike are present but off by default

The entire distractor-and-spike apparatus is described here as a capability. The current leaderboard configuration disables spike-triggered entry, so in that operating point distractor mode is not exercised. It remains fully implemented and can be enabled by turning the spike rejection switch on.

PART 11

Spike rejection — the trigger that guards identity

Goal. Detect the single-frame "the box just teleported" event that betrays a distractor swap, and respond by snapping back and entering distractor mode — *before* the tracker memorises the impostor. Spike rejection is a sensor; its only useful output is to hand the wheel to distractor mode.

11.1 The signal: a camera-compensated normalized step

The core quantity is how far the box centre moved this frame, after subtracting camera motion, normalized by the box diagonal — so the same physical jump means the same thing for a large and a small target:

$$S_t = \frac{\|\Delta \mathbf{p}_t - \Delta \mathbf{p}_t^{\text{cam}}\|}{\text{diag}(b_t)}, \quad \text{ratio} = \frac{S_t}{\text{median}(S_{t-W:t})}$$

A frame triggers when the step is large in absolute normalized terms *and* large relative to the recent baseline (e.g. several times the median recent step). Two vetoes guard against false positives: a camera-residual veto suppresses jumps explained by ego-motion, and an optional appearance veto suppresses jumps where the jumped-to box still looks like the target. A configurable number of consecutive triggers is required before a watch starts.

11.2 The watch: confirm, settle, or cancel

A trigger does not immediately switch modes — it starts a multi-frame watch that resolves one of three ways:

- **Settled switch** — motion drops below a settle threshold for several frames: a real switch is confirmed. Store the switched-to descriptor as a distractor, snap the tracker back to the robust pre-spike anchor, and enter distractor mode.
- **Cancelled** — the box returns near the original anchor: it was a transient (a one-frame blur), not a switch. Clear the watch.
- **Timed out** — motion never stabilises within the watch budget: clear the watch and continue.

Guards short-circuit the entire mechanism during a re-entry cooldown, during heavy camera motion, or in tiny-object mode — three regimes where the normalized step is unreliable and would generate false switches. The design intent is captured in one line: *the spike watcher decides a switch happened, snaps to safety, and hands the steering wheel to distractor mode to sort out which object is real.*

PART 12

Synthesis: how the pieces compose, and the design principles

Having seen each component, it is worth stepping back to the shape of the whole. SiamRAM is not a pipeline of independent tricks; the pieces are coupled by a few recurring principles, and seeing them is what lets you predict the system's behaviour in a situation this report did not enumerate.

12.1 The recurring design principles

- **Separate the three failures.** Occlusion, distractor, and spike are distinct events with distinct responses. The architecture refuses to collapse them into a single low-confidence branch — that separation is the central thesis.
- **Distrust the camera first.** Heavy ego-motion suppresses occlusion entry, distractor entry, and spike detection, because surprising behaviour during a pan is more likely an artefact than an event.
- **Hold, don't gamble.** Every mode prefers to ride a motion prediction for a bounded window over making an irreversible identity decision on weak evidence — below-gate holds, overlap locks, ambiguity holds, reacquire confirmation, and the no-timeout occlusion stance are all the same instinct.
- **Freeze learning under doubt.** The instant identity is in question, template updates and test-time adaptation stop, and memory admission pauses after risky episodes — so the system never teaches itself the wrong object.
- **Calibrate to the target, not to a constant.** The acceptance margin, the template rate, the effective loss threshold, and the spatial tolerance all adapt to what the system measures about this target and this motion.
- **Learn the negatives.** Known distractors are stored and actively subtracted from candidate scores — the system remembers what fooled it.

12.2 How a single hard episode flows through the system

Consider a target that a similar object passes in front of, briefly occluding it. The camera subsystem confirms the camera is steady, so guards do not block. As the impostor crosses, SiamABC's score dips; the hysteretic test and EKF rebuild kick in, skipping the corrupted last frames and extrapolating clean motion. The detectability verdict — decided frames earlier — routes recovery: if the target's class is YOLO-detectable, candidate collection gathers both the impostor and (when it re-emerges) the true target; the DRM composite, with the impostor already in the negative bank from Phase 1, scores the true target higher; SiamABC verification confirms it; reacquire-confirmation guards against a fluke; and commit restores normal tracking with learning re-enabled. The same EKF that held the position throughout is corrected to the recovered box. No single component solved this — the motion model, the camera estimate, the negative memory, the composite score, and the verification gate each contributed one necessary piece.

The mental model to keep

A fast, myopic appearance optimiser in the centre; a motion model that always knows where the target *should* be; an identity memory that knows what it *looks like* and what its impostors look like; and a state machine that, every frame, decides whether to trust the fast tracker, arbitrate, or search — while refusing to gamble the identity on weak evidence. Hold that picture and every parameter in Part 13 has an obvious place.

PART 13

Parameter reference

A compact map of the dials that change behaviour, grouped by the part of this report that explains them. Values shown are illustrative current defaults; the configuration file is the authoritative source.

Motion, camera, and memory

Parameter	Default	Effect
homography_mode	classic	Camera-motion estimator: classic / accurate / botsort (Part 5)
gmc_prior_enabled	true	Warp the previous box into SiamABC's search prior (Part 5.2)
camera_motion_heavy_*	20 px / 0.3	Absolute and normalized heavy-motion gates (Part 5.3)
mem_capacity	50	Size of the RAM appearance buffer (Part 6.1)
drm_tau_sim / drm_mmin	0.75 / 3	Promotion: similarity bar and supporting-vote count (Part 6.2)

Re-acquisition composite and adaptive controllers

Parameter	Default	Effect
drm_lam_app ... lam_mot	1, 0, 0, 0, 0, 0	Weights on appearance / IoU / dist / dir / time / motion (Part 6.3)
drm_gamma	0	Negative-bank suppressor weight (Part 6.3)
drm_margin	auto	Acceptance margin; "auto" enables the adaptive estimator (Part 7.1)
drm_margin_auto_{min,max,delta}	0.5 / 0.8 / 0.05	Clamp range and offset below the self-score EMA (Part 7.1)
template_rate_auto_enabled	false	Glide N / window from a motion EMA (Part 7.2)

Occlusion lifecycle

Parameter	Default	Effect
conf_threshold / entry_patience	0.5 / 1	Loss score and hysteresis frames before entry (Part 9.1)
no_occlusion_first_n_frames	18	Warm-up grace before any loss can be declared (Part 8.3)
yolo_detectability_enabled	true	Run the early probe that selects the recovery strategy (Part 9.4)
occ_siam_reacq_threshold	0.8	Phase-0 SiamABC score gate, Gate A (Part 9.3)
app_match_threshold	0.6	Phase-0 DRM appearance gate, Gate B (Part 9.3)
cand_collection_frames	1	YOLO collection frames; >1 enables per-candidate velocity (Part 9.3)
reacq_threshold / confirm_frames	0.7 / 1	Verification bar and confirmation frames before commit (Part 9.5)

Distractor and spike (present; disabled in the current config)

Parameter	Default	Effect
spike_reject_enabled	false	Master switch for jump detection → distractor entry (Part 11)
spike_reject_ratio	2.5	Step must exceed this × the baseline to trigger (Part 11.1)
distractor_mode_selected_min_similarity	0.9	Accept gate for the winning candidate (Part 10.2)
..._below_gate_hold_frames	15	Bounded motion-only hold below the accept gate (Part 10.2)
..._below_gate_force_occlusion	true	Hand off to occlusion instead of reverting to base tracker (Part 10.2)
mahalanobis_threshold	9.21	χ^2 motion gate on candidate centres (Part 10.1)

This report describes the SiamRAM robustness layer at the level of theory and intent. The source modules and the inference configuration are the authoritative description of runtime behaviour; where this document and the code disagree, the code is what runs.